

A Lean-checked finite bound for unimodality of forest independence polynomials

Author: Rich Patterson. **Manuscript date:** 28 September 2026. **Status:** finite theorem checked locally; external peer review not established.

Abstract

Every finite simple forest with at most 37 vertices has a unimodal independence sequence. We prove this finite bound by combining actual graph counting, complete rooted-state closure through order 18, exact interval certificates for connected orders 19–37, and a log-concavity convolution argument for disconnected forests, without an inherited census assumption or unproved certificate premise. The formalization was checked in Lean 4.30 with Mathlib using the custom kernel-assembly and audit procedure described in Section 6.

1. Statement and mathematical context

For a finite simple graph G , let $i_k(G)$ denote the number of independent vertex sets of cardinality k . Its independence polynomial is

$$I_G(x) = \sum_{k \geq 0} i_k(G) x^k.$$

The coefficient sequence is unimodal if there is an index m such that

$$i_k(G) \leq i_{k+1}(G) \quad (k < m), \quad i_{k+1}(G) \leq i_k(G) \quad (k \geq m).$$

Equal adjacent coefficients are allowed. We use ordinary coefficients throughout, without factorial normalization.

Theorem 1. Every finite simple forest G with $|V(G)| \leq 37$ has a unimodal independence sequence.

The empty graph, isolated vertices and disconnected forests are included. In the project's terminology this is “floor38”: any counterexample would have at least 38 vertices. It does not assert the existence of a counterexample of order 38.

The question for trees and forests appears in Problem 3 of Alavi, Malde, Schwenk and Erdős [1]. Earlier computational groundwork includes Reynolds's tree computations [2,3] and the SciNet forest computations [4]. The project's earlier baseline review used their recorded finite ranges; the theorem presented here proves its own smaller-order base and does not import those censuses as hypotheses.

The certificate method comes from this project's commissioned research round, *FLOOR208* — *complete finite-difference box exclusion* [5], documented by its saved prompt, returned reports and archive intake. The individual producing seat/model is not recorded. Before formalization, the catalogue was regenerated and the complete certificates replayed with the round's separate verifier; the original production search was not rerun. The present formalization reuses that method and data, with the project-round credit retained.

The disconnected step uses the classical unimodality-preserving convolution principle associated with Keilson and Gerber [6]; Sagan [7] states a finite polynomial formulation. This should be distinguished from the log-concavity product result in the classical literature [8], and from the false assertion that arbitrary products of unimodal polynomials are unimodal.

Two contemporary works must also be distinguished from this finite result. Zhang and Li's manuscript dated 27 September 2026 claims unimodality for all forests [11]; its claimed general theorem and companion computations have not been validated here. For $n \leq 37$, Theorem 1 establishes the same conclusion as their Proposition 1.2, which states unimodality for forests through order 60, using a method independent of the Zhang-Li argument. Fang, Lu, Nevo, Yao and Zheng address sufficiently large forests [12]. We have not built or audited their formalization, nor proved that its range combines with Theorem 1 to cover every order. No priority claim, unrestricted conclusion or assessment that either external work is invalid follows from this paper.

2. Formal meaning of the graph statement

The checked target is

```
ErDOS993.Floor216.actual_forest_unimodal_through_thirty_seven :
  ∀ n ≤ 37, ∀ G : Graph n, IsForest G → Unimodal (coefficient G)
```

ForestStatements.lean represents Graph n by a symmetric, loopless Boolean adjacency matrix on Fin n . IsForest excludes every injective cycle of length at least three. This is an actual finite-graph statement, rather than a theorem about an abstract polynomial grammar assumed to represent graphs.

coefficient G r enumerates independent subsets of cardinality r . The subset-coverage and nonduplication lemmas identify this enumeration with counting vertex subsets; independent_iff identifies its Boolean predicate with pairwise nonadjacency. coefficient_above_order proves zero coefficients beyond the vertex count. Thus the formal predicate agrees with the sequence in Theorem 1, including its zero tail.

The rooted and interval calculations below are connected to this graph definition by proved recurrence, component, relabeling and counting lemmas. A successful scalar calculation alone would not supply that connection.

A separately checked corollary now states the same theorem directly for Mathlib: for every finite vertex type V and $G : \text{SimpleGraph } V$, if $\text{Fintype.card } V \leq 37$ and $G.\text{IsAcyclic}$, then $k \mapsto (G.\text{indepSetFinset } k).\text{card}$ is unimodal. Its name is

ErDOS993.ForestCore.mathlib_forest_unimodal_through_thirty_seven. Ordinary Lean proofs establish exact acyclicity and counting equivalences and transport under graph isomorphism. The concrete application supplies the completed theorem above; no assumed finite-bound theorem remains in the corollary. This result uses only the same three standard axioms.

3. Rooted states and complete finite closure

For a rooted tree (T, v) , retain both boundary polynomials

$$R(T, v) = (I_T, I_{T-v}).$$

Let deleting the root leave rooted branches (T_j, v_j) , with states (P_j, Q_j) . Counting independent sets according to whether they contain the root gives

$$I_T = \prod_j P_j + x \prod_j Q_j, \quad I_{T-v} = \prod_j P_j. \quad (1)$$

The first product counts sets excluding the root. A set containing the root cannot contain any adjacent branch root, which gives the second product with the factor x . Products are valid because distinct branches have no edges between them.

`Floor216GraphPolynomial.lean` proves vertex deletion and multiplication across separated vertex sets directly from independent-set counting. `actual_rootState_attach` in `Floor216BankCompleteness.lean` proves (1) for actual root-deletion components. The components have smaller order, and the forest/tree properties needed for the induction are proved graph properties.

Lemma 2 (bank completeness). If a bank of rooted polynomial states is closed under the attachment operation (1) through order N , then it contains the state of every rooted tree of order at most N .

Proof. Induct on the vertex count. For a rooted tree, remove its root. Every rooted branch has smaller order, so its state occurs in the bank by induction. Attachment closure then places the original tree's state in the bank. The empty branch list supplies the one-vertex case. The statement requires coverage, not uniqueness: additional bank entries do not invalidate it. This is the theorem `rooted_bank_complete_through`. ■

The concrete bank has checked attachment closure through 18 and checked ordinary log-concavity of every first-channel polynomial in that range. Schematically, writing `B` for the concrete bank, the results have the form

```
closed18 : BankClosedThrough 18 B
lcThrough18 : ∀ n ≤ 18, ∀ s ∈ B n, LogConcave (intCoeff s.1)
```

These are explanatory names, not literal exported signatures. The corresponding completed modules are `Floor38CodedClosed18` and `Floor38CodedLCComplete18`. The tables' indices 0–17 store bundles to which a root is added: `rootBank (n+1)` represents rooted order $n + 1$. The largest covered rooted order is therefore 18.

The coded-bank theorems establish membership, attachment closure, coefficient bounds and the correspondence between codes and polynomial states. Catalogue completeness is a conclusion of Lemma 2 and these checked closure facts; it is not an axiom attached to an external enumeration.

For a sequence q , ordinary log-concavity means

$$q_{k-1}q_{k+1} \leq q_k^2.$$

Coefficients are extended by zero outside their support. Independence-polynomial coefficients are nonnegative and have no internal zeros: every subset of an independent set is independent. The formal results `actual_graph_no_holes` and `actual_graph_unimodal_of_lc`

supply this connection. Consequently every tree through order 18 has a log-concave, hence unimodal, independence sequence.

4. Reduction of connected orders 19-37

Choose a centroid of a tree of order n . Each root branch has at most $h = \lfloor n/2 \rfloor$ vertices. The proved grouping argument collects branches into two or three groups of size at most h , apart from a precisely specified exceptional case: four groups, each consisting of a single rooted branch of size $h/2$, with n odd and h even. “Single” here refers to the number of branches in a group, not to a one-vertex branch.

Grouping preserves the products of both boundary channels in (1). The typed bank keeps the distinction between an individual rooted branch and a bundle, including the admissible size bounds for each. This information is part of the formal coverage argument, rather than being discarded when states with equal polynomials are compared.

`actual_tree_case_shape` proves the graph reduction. `supplied_cases_complete` and the sorted-case transport prove that all admissible group-size patterns occur among the supplied cases. When $n \leq 37$, $h \leq 18$, so the verified rooted-state closure suffices for the reduction. The exact order certificates `supplied_order19` through `supplied_order37` then discharge the polynomial obligations for every connected case in that range.

4.1 Finite-difference interval certificates

For grouped channels, write

$$J(x) = \prod_j P_j(x) + x \prod_j Q_j(x).$$

The predecessor method [5] computes rigorous enclosures after cancellation. With $D(P) = (1 - x)P$, and nonnegative integers d_j summing to d ,

$$D^d J = \prod_j D^{d_j} P_j + x \prod_j D^{d_j} Q_j. \quad (2)$$

This identity distributes an exact common factor; it does not assume independence between interval errors. Different valid allocations in (2) can yield different enclosures for the same coefficient, and intersecting them retains a valid enclosure. A common positive normalization of the two channels of each factor preserves the signs relevant to unimodality.

To fix the indexing convention, put $a_k = \lfloor x^k \rfloor J$ and use **departure differences** $\delta_k = a_{k+1} - a_k$. The coefficient of x^{k+1} in $(1 - x)J$ equals δ_k . Some earlier reports use the corresponding arrival index instead; the formal graph acceptance theorem fixes the translation.

The acceptance rule establishes nonnegative departure differences before a window, nonpositive differences after it, and nonincreasing differences within it. These conditions exclude a negative difference followed later by a positive difference, and hence imply unimodality, including plateaus. First- and second-difference interval bounds provide the sufficient inequalities. Signed rounding, interval multiplication and convolution are proved sound, and positive scaling preserves the conclusion.

The certificate tree covers the Cartesian product of the complete factor catalogues. Its split-coverage proofs account for the full parent range, and every accepted terminal supplies the required valid enclosures and acceptance inequalities. The mathematical requirement is complete checked coverage, not successful behavior on sampled states or trust in the search heuristic that found the split tree.

`graph_certificate_sound` converts accepted interval certificates into graph unimodality. `connected_tree_from_bank_certificates` connects that algebra with the actual connected trees supplied by the structural reduction. `FinalCombiner.named_cases` combines the nineteen proved order results without weakening their statements. Thus every tree of order at most 37 is covered: orders through 18 by log-concavity, and 19–37 by complete connected certificates.

5. Disconnected closure and proof of Theorem 1

We use the following finite-support convolution principle in its required form: convolution of a unimodal sequence with a nonnegative log-concave sequence having no internal zeros is unimodal [6,7]. The local development proves the version used by the forest argument. A reference to the classical theorem is mathematical credit, not a new Lean axiom.

Proof of Theorem 1. Proceed by strong induction on the number n of vertices. The empty graph is immediate. For a connected nonempty forest, the graph is a tree, and Sections 3–4 provide the result.

Suppose the forest is disconnected and nonempty. It has a nonempty connected component C with $2|C| \leq n$: choose a smallest component among at least two components. Since $n \leq 37$, we have $|C| \leq 18$. Section 3 proves log-concavity of I_C , and its coefficients are nonnegative with no holes. The complementary induced forest H has fewer vertices, so I_H is unimodal by induction. There are no edges between C and H , giving $I_G = I_C I_H$. The convolution principle proves unimodality of I_G . ■

`small_connected_component`, `whole_graph_lc_split` and `forest_floor_from_connected` implement this argument, including actual component extraction and preservation of the forest property. The same induction gives the entire forest base through 18. `floor38_from_lc_base` combines that proved base with the connected certificates; the final size condition is $37 < 2(18 + 1) = 38$.

In particular, the proof does **not** retain an assumption that all forests through 30, 31 or 32 were already verified. The earlier computational baseline is historical context, not a remaining formal premise.

6. Kernel assembly and evidence

The final mathematical assembly supplies the concrete bank, its checked closure and log-concavity results, and all nineteen connected-order theorems to the general forest theorem. The actual implementation of this last step needs a precise disclosure because the complete import environment exceeded the available memory for the original ordinary workflow.

The native helper `FinalExactKernelAssembler.lean` constructs a subset dictionary containing **exact original ConstantInfo objects and their proof bodies**. It retains the unchanged full original module data and imports. It verifies dependency closure, original declaration

ownership and axiom-cache coverage; a missing cache entry requires closure through the theorem's proof body. It does not replace theorem declarations with axioms. New final proof terms, including the bank-equality arguments, are submitted to Lean's ordinary kernel declaration check with `debug.skipKernelTC=false`.

The helper exports the actual `Floor38FinalCandidate.olean`. A fresh native process reimports that object using the **same custom exact loader**, checks that the target is a theorem of the literal independently compiled type in `FinalKernelExpected.lean`, checks the absence of universe parameters, and submits a new alias to the kernel. The ill-typed proof-of-False negative control is rejected. This completed full check must not be described as an ordinary full Lean reimport. A separate **small pilot** did pass ordinary reimport and comparison with standard finalization.

Because selecting a declaration dictionary does not itself reproduce all of ordinary full-environment finalization, a separate complete original import-union check is required. `ExactKernelImportUnionAudit.lean` checks duplicate-name compatibility using Lean's merge relation. The completed run covered 22,560 modules and 8,322,439 constants, found 378 compatible duplicate occurrences and zero conflicts, and resolved narrowed-hash collisions by full names. Its incompatible-duplicate and distinct-name/hash-collision controls passed.

The full local provenance audit then checked 17,486 reachable project modules: 17,389 floor modules and 97 external project modules, including all four cross-project bridges. It reconciled source/object/compiler/setup and dependency evidence, the exact target, final export/reimport and compatibility receipts, and final file stability. The reported axiom set is exactly

```
propext
Classical.choice
Quot.sound
```

An axiom printout alone would not establish Theorem 1. The graph definitions, literal target, dependency provenance, actual enabled kernel checks and custom-loader semantics are also necessary to interpret the result. Installed, revision-locked Lean and Mathlib compiled libraries are an explicit upstream trust boundary; a complete local rebuild of all upstream sources is not claimed. Lean and Mathlib are credited in [9,10]. No independent external peer-review certification is claimed.

7. Reproduction and reusable interfaces

The complete source release contains the audited ordinary proof-source closure and embedded finite data, the native tools, dependency locks, explanation and portable runner. It is a **source** release; the original project's full proof evidence is distinct from a fresh build on another machine. The public evidence summary records relative source paths and links original local receipts by hashes. It is not presented as a newly generated compiler receipt.

Fixed artifact	SHA-256
Completed local final audit	323a4c545e6867718d24d2fc0f6f0002497d65f7d1ec70b2a1842f78ae2414e9
Final saved Lean object	36fe469430177481fd6a79ec88b200a2b3dfa4b210ad33fa2aac4769d7c2d203

Complete source ZIP

c0d65a24e6eb5fbe09f4ee3d00a1283dbc72a458429c5c6bd5486abc66e65960

The archive `floor38_unconditional_lean_sources_2026-09-28.zip` has 17,501 verified entries and size 597,672,786 bytes. Source-byte matching and compressed-entry read-back passed. These are packaging checks, not another proof compilation. The toolchain is Lean 4.30.0, commit `d024af099ca4bf2c86f649261ebf59565dc8c622`; Mathlib and additional package revisions are pinned in the supplied lock.

With installed pinned dependencies, the archive's workflow is

```
python portable_build_floor38.py
python portable_build_floor38.py --execute --mode native --toolchain /path/to/lean-toolchain
--mathlib-project /path/to/mathlib-project
```

The first command checks configuration and prints a plan. The second builds the ordinary project modules, then performs native assembly, fresh custom-loader reimport and complete compatibility checking. A complete fresh execution of that exported workflow has **not** been run. Its optional conventional source under `standard/` is a distinct uncompiled alternative; it includes an extra `certifiedFloor38Bank` definition absent from the checked native object. Ordinary full final-source elaboration is therefore also not claimed.

The optimized final native assembly used under 1 GiB private memory locally. This does not describe the full rebuild's memory requirement: large ordinary dependency compilations exceeded 6 GiB, and one order 36 attempt reached about 7.14 GiB before an 8 GiB-cap retry. The historical import-object inventory exceeded 89 billion bytes. No full portable runtime guarantee is available.

A separate small `ForestCore` package extracts 108 preserved general modules and supplies Mathlib-facing graph/counting, acyclicity, finite-vertex transfer and sequence interfaces. Its complete ordinary Lake build passed in a fresh source directory: 116 project modules, 735.526 seconds, with no copied project proof objects or custom assembler. Installed locked upstream caches were reused; this was not an independent-machine rebuild of the full finite proof. Its transfer theorem explicitly takes a proved finite bound as input and does not itself supply a concrete finite census. The sequence/counting interfaces were compared with the formulations inspected in [12]; no theorem from that external formalization was imported or certified. A generic linear-certificate soundness theorem is also reusable algebra, not verification of another paper's complete finite inputs or graph reduction.

The concrete Mathlib corollary was checked in a separate environment using the original audited graph foundation, avoiding replacement by separately rebuilt objects. Its new proof modules compiled ordinarily. The final application passed kernel checking, export, fresh custom-loader reimport with a checked alias, literal target/universe checks and the invalid-proof negative control. The enlarged complete import-union audit covered 22,603 modules and 8,327,983 constants, with 385 compatible duplicate occurrences and zero conflicts. The new saved theorem's SHA-256 is

`b6095b384e72725e98dd74fca15ced81e6a834c3fb615237924754f25ea826df`. This extends the interface; it does not claim a fresh rebuild of the old certificate corpus or an ordinary full final import.

8. Contributions, credit and remaining publication matters

The project contributes a checked connection from actual graph counting to the complete finite-state and interval-certificate argument, removal of the historical inherited census premise, the concrete finite proof assembly, and its recorded provenance. The certificate approach was developed in the project's FLOOR208 research round [5]. The earlier computational and classical contributions are credited above; this description does not claim that every mathematical ingredient originated here.

Development involved substantial generative-AI assistance in mathematical reasoning, formalization, programming, debugging, audit orchestration and writing. Rich Patterson directed the project and is the author and publication maintainer. AI-assisted checks and separate AI seats are not represented as independent human refereeing. The proof claim rests on the stated formal evidence and trust boundary, not on a model's assurance.

The scoped release-license decision remains to be completed. FLOOR208's project origin is established; its missing individual seat/model metadata does not make it an unidentified outside contribution. Existing upstream notices are preserved. The proposed license for original project contributions is Apache-2.0 with the covered files specified explicitly.

The theorem excludes counterexamples through 37. No Lean-verified ceiling, higher unaccepted floor, unrestricted resolution of Erdős 993, exhaustive novelty claim or external endorsement is asserted.

References

1. Y. Alavi, P. J. Malde, A. J. Schwenk and P. Erdős. *The Vertex Independence Sequence of a Graph Is Not Constrained*. Congressus Numerantium 58 (1987), 15–23. [Original paper](#).
2. Brett Reynolds. *Mean bounds, structural reductions, and exhaustive verification for tree independence polynomial unimodality*. [Zenodo record 19100781](#).
3. Brett Reynolds. Later tree-census artifacts, results/lc_census_20260814, [commit b85d5dc01bdcaf3d6853147b28c350d74d3ede21](#). The pinned later records cover orders 27–32 with 64 shards per order and zero unimodality alarms; the earlier manuscript [2] advertises coverage through 29. These records were audited, not newly rerun or formalized as a Lean census here.
4. SciNet/scinet-ai contributors. erdos-993-forests computational report and artifacts, [commit fafb35784d4235c9e5dd701fd3b2c1f4955ae9ec](#).
5. Erdős 993 project, FLOOR208 research round. *FLOOR208 — complete finite-difference box exclusion*. Internal report and reproduction materials; local acquisition/replay audit dated 15 September 2026. Individual producing seat/model not recorded. See attribution ledger.
6. J. Keilson and H. Gerber. *Some Results for Discrete Unimodality*. Journal of the American Statistical Association 66(334) (1971), 386–389. [DOI](#).
7. Bruce E. Sagan. *Compositions inside a rectangle and unimodality*, author-hosted manuscript, 5 May 2008, Proposition 2.1. [Paper](#).

8. S. G. Hoggar. *Chromatic polynomials and logarithmic concavity*. Journal of Combinatorial Theory, Series B 16(3) (1974), 248–254. [DOI](#).
9. Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language (System Description)* (2021). [Paper](#).
10. The mathlib Community. *The Lean mathematical library*. [arXiv:1910.09336](#); related publication [DOI 10.1145/3372885.3373824](#).
11. Tong Zhang and Wei Li. *Unimodality of Forest Independence Polynomials*. Zenodo, version 27 September 2026. [DOI: 10.5281/zenodo.22999166](#).
12. Ethan X. Fang, Junwei Lu, Eran Nevo, Yuan Yao and Hailun Zheng. *Unimodality of Independence Polynomials for Sufficiently Large Forests*. [arXiv:2609.20961v1](#), 17 September 2026. The inspected [formalization interface](#) was not built or audited as a complete project here.

Ordinary source names above refer to the complete archive's `src/` tree, native helpers to `tools/`, and the alternative final source to `standard/`. The separate core package and its receipts have their own status. The companion bibliography, BibTeX entries, verification account and reviewer checklist give source and review details without changing the frozen proof artifact.